

個人製作ゲーム 参考資料

東京理科大学創域理工学研究科情報計算科学専攻

修士1年 馬亮宇

自己紹介

氏名: 馬 亮宇

所属: 東京理科大学大学院創域理工学研究科情報計算科学専攻 修士1年

大学院における専攻: 画像認識、自然言語処理

研究テーマ: 「漫画擬音語の日中双方向のマルチモーダル翻訳AIの開発」

ゲーム開発に使用する言語・ツール: C++ (個人製作)、unityとC# (チーム製作)

作品紹介

タイトル:「もじうおーず」

ジャンル:2D見下ろし型タワーディフェンスゲーム

製作人数:1人

製作期間:5カ月

所要時間数の概算:200時間

対応機種:PC(WindowsまたはLinux)

製作意図:

- ①大学の文化祭のサークル企画にてゲームを展示し楽しんでもらうため
- ②ゲームエンジンに頼らないフルスクラッチ開発で、C++の実装力を磨くため

作品紹介

ゲームの各種リンク

ゲームのGitHubのリンク: <https://github.com/RatHorse200/MojiWars>

ソースコード一覧: <https://github.com/RatHorse200/MojiWars/tree/main/src>

ゲーム本編のプレイ: <https://github.com/RatHorse200/MojiWars/releases>

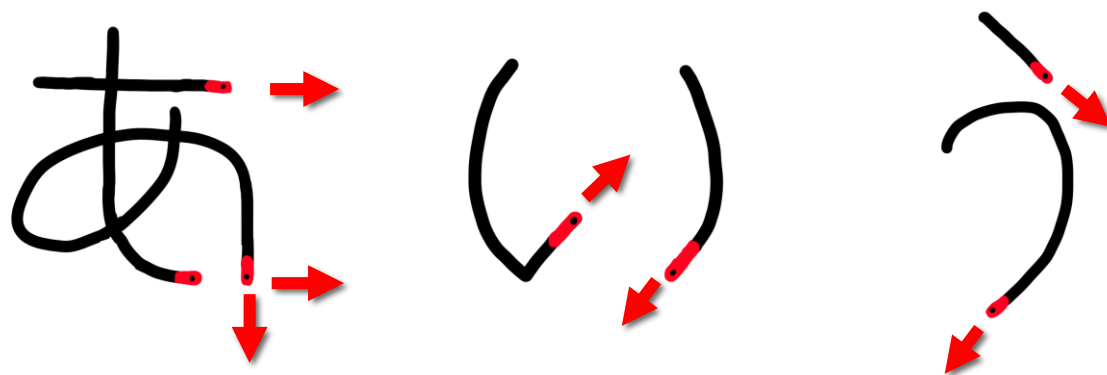
ゲームの紹介動画(1分36秒): <https://www.youtube.com/watch?v=mYgpoLSNX4I>

以降のスライドでは、作品のアピールポイントや開発ツール紹介のスライドの下部に、該当するソースコードへのリンクを記載しております。ぜひあわせてご参照ください。

作品紹介

ゲームのコンセプト

- ①一プレイ5分～15分で、ミニゲームとして丁度いい時間とします。
- ②十代を中心とした幅広い年齢層を対象に、分かりやすいルールと認知負荷のバランスを取ります。
- ③マウスとタイピングの簡単な操作で、誰にでも身近な「ひらがな」を武器として駆使し戦います。



文字を書く「方向(赤矢印)」に弾が発射され、敵を倒します。

作品紹介

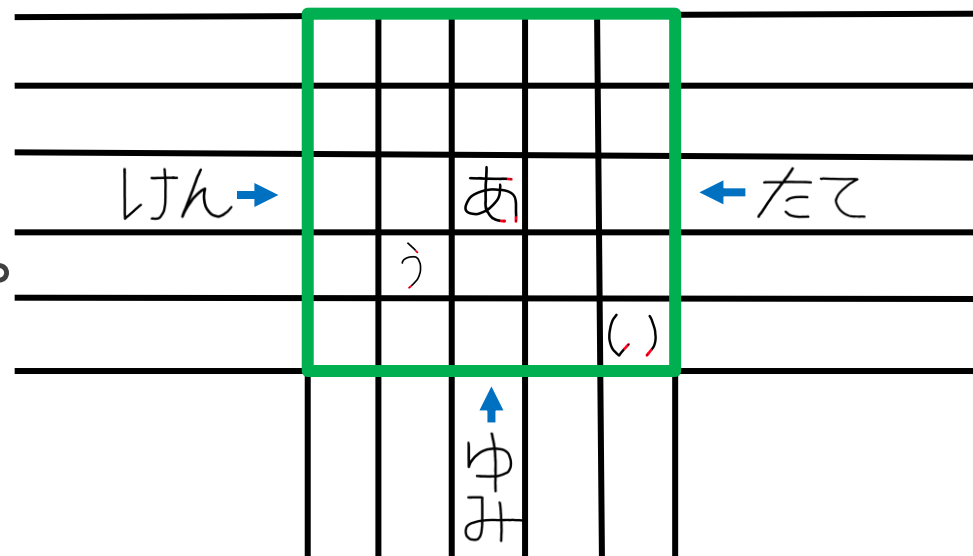
ゲーム説明

左・右・下の3方向から3種類の
迫る敵(けん・たて・ゆみ)を迎え撃ちます。

正面からの攻撃に強い「たて」や
文字を消す弾を放つ「ゆみ」といった
敵の性質や位置を瞬時に把握し、

「この敵には斜めから弾を当てたいから、この位置にあのひらがなを置こう」と
いった**戦略的かつ瞬時の判断**が求められるゲームです。

簡略化したゲーム画面
(緑枠の内側にひらがなを置くことで敵を迎え撃ちます)



作品のアピールポイント

①ゲーム内フォントの自作

・問題点

弾の発射角(上を除く45度刻みの7方向)と、ひと筆の終端方向がズれると、見た目と実際の**挙動が噛み合わず**、プレイヤーの没入感を削いでしまいます。

い

MS Pゴシック 本文

()

私が制作したフォント

・解決法

全ての文字を自作することで、弾の発射角を明確に文字デザインと合わせます。

関連アセットファイル: https://github.com/RatHorse200/MojiWars/tree/main/assets/red_ch

作品のアピールポイント

②テクスチャキャッシュの実装

・問題点

ゲームの処理が重い。

・解決法

テクスチャのGPU転送を初回のみ実行し、二回目以降は**キャッシュ**を返します。

```
// 赤文字の画像データを取得する関数
Texture2D GetRedSpriteTexture(const std::string& romaji) {
    if (romaji.empty()) {
        Texture2D empty = { 0 };
        return empty;
    }
    // キャッシュにあればそのまま返し、なければロードして登録してから返す
    if (redSpriteTextures.find(romaji) == redSpriteTextures.end()) {
        redSpriteTextures[romaji] = LoadRedSprite(romaji);
    }
    return redSpriteTextures[romaji];
}

// 黒文字の画像データを取得する関数
Texture2D GetBlackSpriteTexture(const std::string& romaji) {
    if (romaji.empty()) {
        Texture2D empty = { 0 };
        return empty;
    }
    // キャッシュにあればそのまま返し、なければロードして登録してから返す
    if (blackSpriteTextures.find(romaji) == blackSpriteTextures.end()) {
        blackSpriteTextures[romaji] = LoadBlackSprite(romaji);
    }
    return blackSpriteTextures[romaji];
}
```

関連ソースファイル: <https://github.com/RatHorse200/MojiWars/blob/main/src/sprites.cpp>

作品のアピールポイント

③敵AIの実装(適応型難易度)

状況をスコアリングし、
敵の出現頻度を最適化するAIです。
プレイヤーの敵撃破速度に応じて
出現頻度を変化させ、
初心者も熟練者も楽しめる
難易度を維持します。

```
// 敵が撃破されたことをAIに通知し、適応型難易度を更新する
void NotifyEnemyKilled(float ttk) {
    // 直近で倒した数 (TTK_WINDOW体) のTTKをローリングバッファで記録する
    aiState.recentTTKs[aiState.ttkIdx] = ttk;
    aiState.ttkIdx = (aiState.ttkIdx + 1) % TTK_WINDOW;
    if (aiState.ttkCount < TTK_WINDOW) aiState.ttkCount++;

    // 記録済み分の平均TTKを計算する
    float sum = 0.0f;
    int n = (aiState.ttkCount > 0) ? aiState.ttkCount : TTK_WINDOW;
    for (int i = 0; i < n; i++) sum += aiState.recentTTKs[i];
    float avgTTK = sum / (float)n;

    // 平均TTKに応じた目標倍率を決定する
    // 撃破が速い (< 8秒) : 倍率を下げて間隔を短くする (難しくなる)
    // 撃破が遅い (> 20秒) : 倍率を上げて間隔を長くする (易しくなる)
    // 普通 (8~20秒) : 倍率を標準に戻す
    float targetMult;
    if (avgTTK < 8.0f) targetMult = 0.85f;
    else if (avgTTK > 20.0f) targetMult = 1.3f;
    else targetMult = 1.0f;

    // 急激な変化を抑えるため、現在値と目標値の中間に緩やかに近づける
    aiState.spawnIntervalMult = aiState.spawnIntervalMult * 0.7f + targetMult * 0.3f;

    // 倍率を0.7~1.5の範囲に制限する
    aiState.spawnIntervalMult = std::max(0.7f, std::min(1.5f, aiState.spawnIntervalMult));
}
```

関連ソースファイル: <https://github.com/RatHorse200/MojiWars/blob/main/src/ai.cpp>

作品のアピールポイント

④敵AIの実装

(ユーティリティベースAI)

フィールド状況をスコアリングし、
敵の出現方向を最適化するAIです。

フィールドの陣形の守りが

手薄な方向を分析し、

重点的に敵を配置して

やりごたえを創出しています。

```
// 指定した方向に対する防御密度を計算する (0.0=無防備、1.0以上=十分な防御)
// 左側: 左上 / 左 / 左下 の合計発射点数
// 右側: 右上 / 右 / 右下 の合計発射点数
// 下側: 左下 / 下 / 右下 の合計発射点数
static float CalcDefense(const GameBoard& board, SpawnSide side) {
    int firePoints = 0;
    for (int y = 0; y < GRID_SIZE; y++) {
        for (int x = 0; x < GRID_SIZE; x++) {
            const std::string& ch = board.cells[y][x];
            if (ch.empty()) continue;
            auto it = charFireConfig.find(ch);
            if (it == charFireConfig.end()) continue;
            for (const auto& fp : it->second) {
                bool counts = false;
                if (side == SIDE_LEFT  && (fp.direction == FIRE_UPPER_LEFT || fp.direction == FIRE_LEFT || fp.direction == FIRE_LOWER_LEFT)) counts = true;
                if (side == SIDE_RIGHT && (fp.direction == FIRE_UPPER_RIGHT || fp.direction == FIRE_RIGHT || fp.direction == FIRE_LOWER_RIGHT)) counts = true;
                if (side == SIDE_BOTTOM && (fp.direction == FIRE_LOWER_LEFT || fp.direction == FIRE_DOWN || fp.direction == FIRE_LOWER_RIGHT)) counts = true;
                if (counts) firePoints++;
            }
        }
    }
    // 盤面上の全ひらがなについて、その発射方向がその辺を向いているものの数を合計し、25で割って正規化する
    return std::min(1.0f, (float)firePoints / (float)(GRID_SIZE * GRID_SIZE));
}

// AIの状態を毎フレーム更新する (ユーティリティスコアのみ)
void UpdateAI(const GameBoard& board) {
    // ユーティリティスコアを計算: 防御密度が低い方向ほど高いスコアになる
    // 最小値0.1を設けることで、満員でも選ばれる可能性をゼロにしない
    aiState.utilLeft  = std::max(0.1f, 1.0f - CalcDefense(board, SIDE_LEFT));
    aiState.utilRight = std::max(0.1f, 1.0f - CalcDefense(board, SIDE_RIGHT));
    aiState.utilBottom = std::max(0.1f, 1.0f - CalcDefense(board, SIDE_BOTTOM));
}

// ユーティリティスコアを確率として使った重み付きランダムで、スポーン方向を決定する
// スコアが高い方向ほど選ばれやすいが、低い方向もゼロにはならない
SpawnSide GetAISpawnSide() {
    float total = aiState.utilLeft + aiState.utilRight + aiState.utilBottom;
    float r = (float)GetRandomValue(0, 10000) / 10000.0f * total;
    if (r < aiState.utilLeft) return SIDE_LEFT;
    if (r < aiState.utilLeft + aiState.utilRight) return SIDE_RIGHT;
    return SIDE_BOTTOM;
}
```

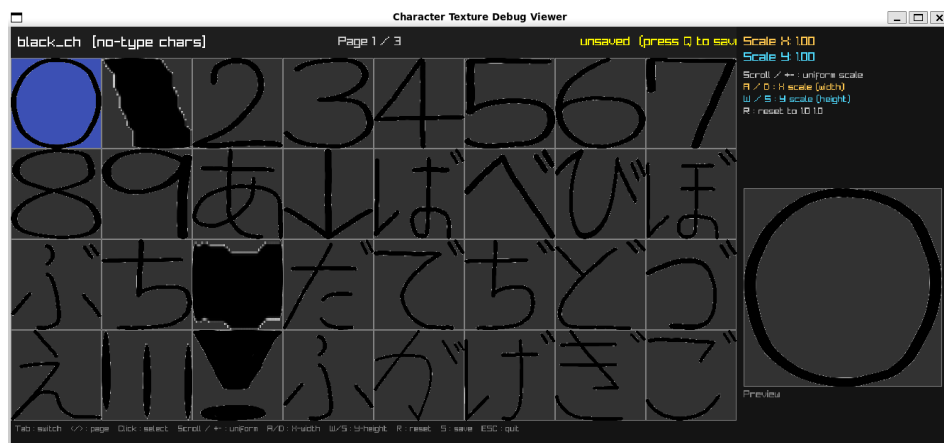
関連ソースファイル: <https://github.com/RatHorse200/MojiWars/blob/main/src/ai.cpp>

製作ツールの紹介

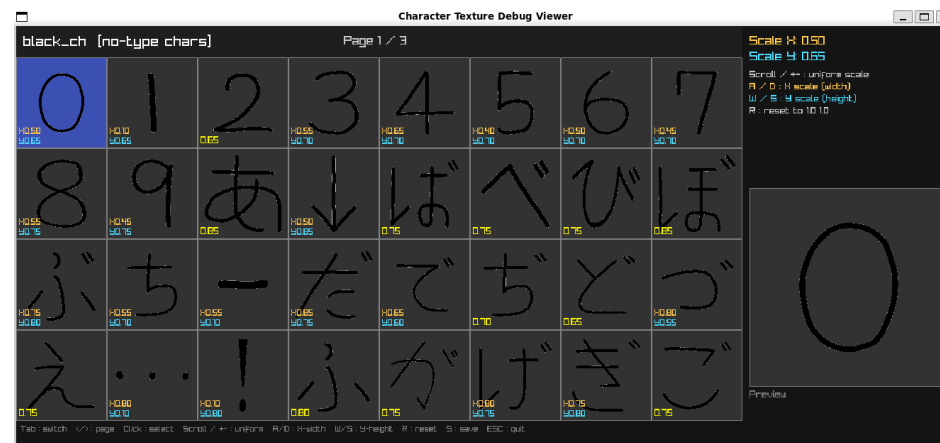
①文字スケール調整ツール

自作フォントの縦横の倍率を個別に調整し、ゲーム画面に**最適なサイズ**で表示させるツールです。

調整前



調整後



調整前: 文字がマスからはみ出し、形が崩れてしまう 調整後: 視認性が上がり、正しく美しい文字として認識できる

関連ソースファイル: https://github.com/RatHorse200/MojiWars/blob/main/src/viewer_editor.cpp

製作ツールの紹介

②発射位置設定ツール

それぞれの文字のビジュアルと弾の軌道が完全に一致するように、ひらがなごとの発射ポイントや角度を視覚的に設定・調整できるツールです。

設定前



この座標から
右上方向に
弾を発射する
という設定

設定後



この座標から
左下方向に
弾を発射する
という設定

関連ソースファイル: https://github.com/RatHorse200/MojiWars/blob/main/src/fire_editor.cpp

今後の目標

①未知の体験を形にする技術

C++の最適化技術とAIの知見で処理の壁を越え、キャラクターがそこで実際に生活しているかのような**生き生きとした世界**を構築します。

②アイデアを加速させる環境

自作ツールのノウハウを展開し、多様な職種の仲間が試行錯誤に集中できる**最高の開発土台**を提供します。

これらの力で、既存の枠にとらわれない**新規グローバルIPの創出**に挑戦します。

最後に

私のゲーム制作に対する思いについて

身近な「ひらがな」の新しい一面で面白さを共有し、
それをきっかけに家族や友達との会話が弾んだり、
遊んでくれたお客さんの**楽しい思い出**として、
記憶にそっと残る作品になれば嬉しいな、
という願いを込めて、このゲームを制作しました。

最後まで読んでくださり、
ありがとうございました。